project_report

March 14, 2026

1 CS 615 Final Project: Recipe Quality Prediction and Semantic Clustering

Author: Ian Auger

1.1 Introduction

Online recipe platforms exhibit a strong positivity skew, where many recipes receive highly favorable star ratings even when critical qualitative feedback is present in the accompanying review text. As a result, important signals such as flavor issues, preparation difficulty, or ingredient problems are often diluted within large volumes of uniformly positive reviews.

Objective. The goal of this project is to build a deep learning system that predicts a stabilized recipe-quality score by combining structured recipe metadata with semantic feedback signals extracted from user reviews.

1.1.1 Phase 1 Retrospective: Data Engineering and Distributed NLP

This project builds directly on a prior phase completed in DSCI 632. In that earlier work, the focus was on transforming unstructured Food.com review text into structured multi-label semantic signals using a distributed PySpark pipeline.

Key components of Phase 1 included: - **Qualitative Tagging via Distributed NLP:** A transformer-based zero-shot baseline was used to generate silver labels, which were then used to support a scalable Word2Vec-centroid tagging system in PySpark. - **Culinary Semantic Representation:** Reviews were embedded in a custom 128-dimensional culinary vector space, enabling the extraction of both common praise signals and rarer critique-oriented signals such as difficulty, dryness, excessive sweetness, or ingredient issues.

The deep learning phase begins where that pipeline ends: with recipe-level aggregates of these semantic review features merged with structured recipe metadata.

[ ]:
```
# Core Libraries
import sys

# Dynamic Path Setup to Ensure src is Importable
from pathlib import Path
p = Path.cwd().resolve()
while p != p.parent and not (p / "src").exists():
    p = p.parent
```

```python
if str(p) not in sys.path:
    sys.path.insert(0, str(p))

# Internal Pipeline Modules
from src.config import load_settings
from src.dataset import RecipeDataset
from src.preprocessing import preprocess_data
from src.visualizations import (
    rating_distribution_plots,
    generate_alltime_leaderboard,
    generate_hp_leaderboard,
    plot_recipe_manifold,
    summarize_inference_metrics,
    plot_regression_diagnostics,
    show_largest_prediction_errors,
    plot_umap_grid,
    calculate_spearmanr
)

# Setup Environment
s = load_settings()
```

1.2 Preprocessing: Target Formulation and Feature Engineering

The deep learning stage begins with a recipe-level table formed by combining original Food.com recipe metadata with aggregated semantic review signals from Phase 1. Before modeling, these raw inputs required additional engineering to produce a numerically stable supervised learning dataset.

1.2.1 Target Engineering: Bayesian Smoothing

Raw average ratings on recipe websites are highly sensitive to low-sample variance. For example, a recipe with a single 5-star review can outrank a recipe with dozens of consistently high ratings simply because its estimate is based on almost no evidence. To reduce this instability, the project uses a **Bayesian-smoothed rating** as the regression target.

This target combines: - **A global prior (C):** the mean rating across the full recipe population - **A confidence threshold (m):** based on the upper quantiles of review counts - **A weighted adjustment:** recipes with few reviews are shrunk toward the global mean, while highly reviewed recipes remain closer to their observed average

This produces a more stable continuous target that is better suited for regression than the raw mean rating alone.

1.2.2 Feature Engineering

To construct the final model inputs, the preprocessing pipeline performs several deterministic transformations:

- **Structured metadata extraction:** Recipe-level structural and nutritional attributes such

as `minutes`, `n_steps`, `n_ingredients`, `calories`, `sodium`, and `protein` are retained as continuous features.

- **Semantic review aggregation:** Review-level NLP outputs from Phase 1 are grouped to the recipe level. Tag prevalence features (`pred_*`) summarize how frequently a semantic signal appears across reviews, while tag intensity features (`intensity_*`) summarize the average strength of that signal.
- **Categorical encoding:** Cleaned recipe tags and ingredients are converted into one-hot encoded indicators for the most common categories, reducing cardinality while preserving interpretable recipe context.
- **Min-max scaling:** Continuous numerical features are rescaled to the `[0, 1]` range so they can be optimized more consistently alongside sparse binary inputs such as category and ingredient indicators.

1.3 Exploratory Data Analysis

Before comparing architectures, it is useful to inspect the dimensionality of the final dataset and evaluate how Bayesian smoothing changes the target distribution. This section summarizes the size of the modeling table and contrasts the raw mean rating distribution with the engineered Bayesian target.

[2]:
```
# Load the preprocessed Phase 1 dataset
df = preprocess_data(s, overwrite_processed=False)
full_dataset = RecipeDataset(df)

# Data
print(f"Total Recipes: {len(full_dataset):,}")
print(f"Metadata Features: {full_dataset.meta_dim}")
print(f"Tag Features: {full_dataset.tag_dim}")
print(f"Continuous Metadata Features: {full_dataset.num_dim}")
print(f"Categorical Metadata Features: {full_dataset.cat_dim}")

df[['raw_mean_rating', 'review_count', 'bayesian_rating']].describe().T
```

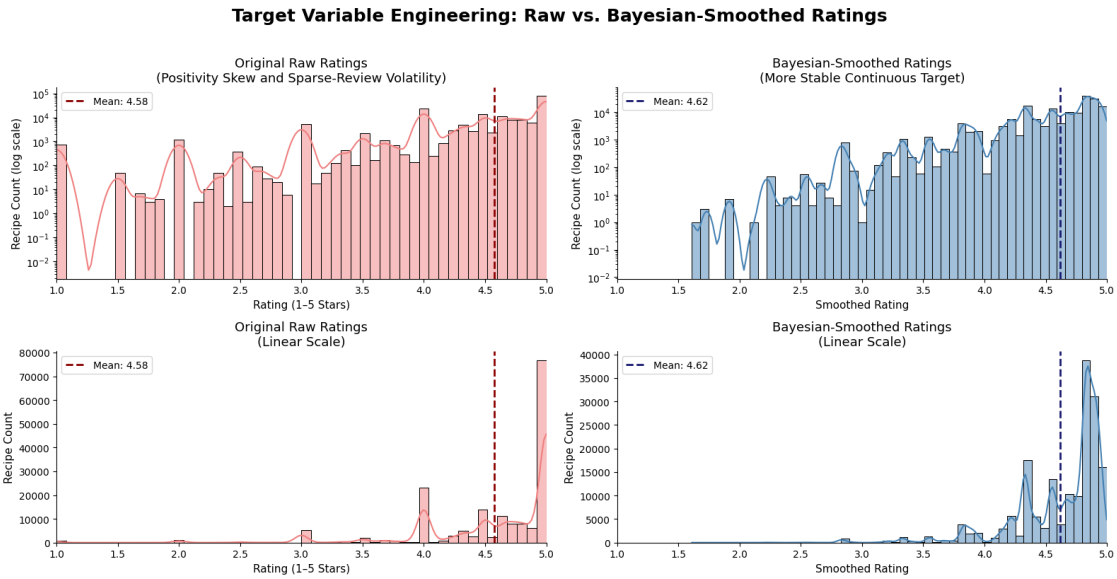
```
File already exists: C:\Users\iauge\Documents\Drexel MSDS\CS
615\Project\data\processed\PROCESSED_recipes.parquet
Total Recipes: 173,550
Metadata Features: 210
Tag Features: 34
Continuous Metadata Features: 10
Categorical Metadata Features: 200
```

[2]:
```
                    count      mean        std      min       25%       50%  \
raw_mean_rating  173550.0  4.577309   0.598100  1.00000  4.333333  4.800000
review_count     173550.0  5.743181  18.822915  1.00000  1.000000  2.000000
bayesian_rating  173550.0  4.616015   0.366879  1.61025  4.415375  4.777707

                      75%          max
raw_mean_rating  5.000000     5.000000
```

```
review_count     5.000000  1496.000000
bayesian_rating  4.887166     4.998246
```

[ ]:
```
rating_distribution_plots(df, raw_col='raw_mean_rating',␣
↪smooth_col='bayesian_rating')
```



1.3.1 Rating Smoothing Analysis

The raw rating distribution shows a strong **positivity skew**, with most recipes concentrated between roughly 4 and 5 stars. The log-scaled view highlights another issue: recipes with very few reviews create scattered low-frequency bins across the rating range, reflecting **high variance from sparse feedback**.

To reduce this instability, the model predicts a **Bayesian-smoothed rating** rather than the raw mean rating. This approach shrinks recipes with few reviews toward the global average while allowing highly reviewed recipes to retain their observed scores.

The resulting target remains positively skewed but is **more stable and less dominated by small-sample noise**, making it a more reliable regression target for the neural network.

1.4 Model Architecture

To model recipe quality, we use a **dual-stream neural architecture** that processes two distinct feature groups:

1. **Recipe Metadata**
   - structural properties (minutes, steps, ingredients)
   - nutritional information (calories, fat, sodium, etc.)
   - categorical recipe tags
   - ingredient indicators

2. **Semantic Review Signals**
   - aggregated qualitative feedback derived from Phase 1 NLP tagging
   - tag prevalence features (`pred_*`)
   - tag intensity features (`intensity_*`)

These two feature families capture fundamentally different information. Metadata describes **what the recipe is**, while semantic review features describe **how users experienced it**.

1.4.1 Dual-Encoder Design

To reflect this distinction, the model first encodes metadata and semantic review features separately. These representations are then fused before producing a final recipe quality prediction.

1.4.2 Architecture Variants

Several architectural heads were tested to study the effect of depth and fusion strategy:

- **Shallow MLP**
  A simple feedforward baseline operating on the fused representation.
- **Deep MLP**
  A deeper fully connected network to capture higher-order feature interactions.
- **Residual Networks**
  Skip-connected blocks designed to stabilize optimization in deeper networks and improve feature reuse.
- **Two-Tower Architecture**
  Separate deep encoders for metadata and semantic tags that are fused only at the final prediction stage.

These architectures share the same preprocessing pipeline and target variable, allowing direct comparison of how architectural design influences prediction performance.

[15]:
```
generate_alltime_leaderboard(s.results_dir, mode="best_per_model_ablation",␣
↪sort_by="test_rmse")
```

```
=== Best Run per Model × Ablation ===
| Run Date      | Architecture   | Ablation     | Loss     |   RMSE |    MAE |
MSE |   Best Val Loss |   Epochs |
|:--------------|:---------------|:-------------|:---------|-------:|-------:|--
-----:|----------------:|---------:|
| Mar 13, 17:03 | deep           | all_features | mse      | 0.2964 | 0.2189 |
0.0878 |          0.0842 |       87 |
| Mar 13, 16:09 | shallow        | all_features | huber    | 0.2964 | 0.2164 |
0.0878 |          0.0412 |       68 |
| Mar 13, 18:44 | residual_v2    | all_features | mse      | 0.2973 | 0.2196 |
0.0884 |          0.0843 |       50 |
| Mar 13, 17:41 | residual       | all_features | huber    | 0.2975 | 0.2162 |
0.0885 |          0.0417 |       54 |
| Mar 13, 20:06 | two_tower      | all_features | huber    | 0.298  | 0.2154 |
```

```
0.0888 |          0.0419 |       80 |
| Mar 13, 19:17 | residual_v3    | all_features | huber    | 0.2997 | 0.2183 |
0.0898 |          0.0419 |       51 |
| Mar 12, 18:06 | residual_v3    | tag_only     | huber    | 0.3029 | 0.2226 |
0.0917 |          0.0413 |       52 |
| Mar 12, 14:40 | shallow        | tag_only     | mse      | 0.3047 | 0.2144 |
0.0928 |          0.0834 |       46 |
| Mar 12, 16:14 | residual       | tag_only     | mse      | 0.3103 | 0.2192 |
0.0963 |          0.0836 |       43 |
| Mar 12, 15:36 | deep           | tag_only     | log_cash | 0.3112 | 0.2234 |
0.0968 |          0.039  |       58 |
| Mar 12, 17:13 | two_tower      | tag_only     | huber    | 0.3148 | 0.2176 |
0.0991 |          0.0424 |       57 |
| Mar 12, 17:09 | residual_v2    | tag_only     | mse      | 0.3161 | 0.221  |
0.0999 |          0.084  |       49 |
| Mar 12, 18:48 | two_tower      | meta_only    | huber    | 0.3732 | 0.2692 |
0.1393 |          0.0419 |       48 |
| Mar 12, 14:28 | shallow        | meta_only    | mse      | 0.3738 | 0.2713 |
0.1397 |          0.0835 |       49 |
| Mar 12, 15:04 | deep           | meta_only    | huber    | 0.3738 | 0.2714 |
0.1397 |          0.0413 |       56 |
| Mar 12, 16:00 | residual       | meta_only    | mse      | 0.3739 | 0.2778 |
0.1398 |          0.0841 |       50 |
| Mar 12, 16:55 | residual_v2    | meta_only    | log_cash | 0.376  | 0.2727 |
0.1414 |          0.0389 |       43 |
| Mar 12, 17:44 | residual_v3    | meta_only    | huber    | 0.3767 | 0.2759 |
0.1419 |          0.0415 |       50 |
```

[15]:

	Run Date	Architecture	Ablation	Loss	RMSE	MAE \
0	Mar 13, 17:03	deep	all_features	mse	0.2964	0.2189
1	Mar 13, 16:09	shallow	all_features	huber	0.2964	0.2164
2	Mar 13, 18:44	residual_v2	all_features	mse	0.2973	0.2196
3	Mar 13, 17:41	residual	all_features	huber	0.2975	0.2162
4	Mar 13, 20:06	two_tower	all_features	huber	0.2980	0.2154
5	Mar 13, 19:17	residual_v3	all_features	huber	0.2997	0.2183
6	Mar 12, 18:06	residual_v3	tag_only	huber	0.3029	0.2226
7	Mar 12, 14:40	shallow	tag_only	mse	0.3047	0.2144
8	Mar 12, 16:14	residual	tag_only	mse	0.3103	0.2192
9	Mar 12, 15:36	deep	tag_only	log_cash	0.3112	0.2234
10	Mar 12, 17:13	two_tower	tag_only	huber	0.3148	0.2176
11	Mar 12, 17:09	residual_v2	tag_only	mse	0.3161	0.2210
12	Mar 12, 18:48	two_tower	meta_only	huber	0.3732	0.2692
13	Mar 12, 14:28	shallow	meta_only	mse	0.3738	0.2713
14	Mar 12, 15:04	deep	meta_only	huber	0.3738	0.2714
15	Mar 12, 16:00	residual	meta_only	mse	0.3739	0.2778
16	Mar 12, 16:55	residual_v2	meta_only	log_cash	0.3760	0.2727
17	Mar 12, 17:44	residual_v3	meta_only	huber	0.3767	0.2759

	MSE	Best Val Loss	Epochs
0	0.0878	0.0842	87
1	0.0878	0.0412	68
2	0.0884	0.0843	50
3	0.0885	0.0417	54
4	0.0888	0.0419	80
5	0.0898	0.0419	51
6	0.0917	0.0413	52
7	0.0928	0.0834	46
8	0.0963	0.0836	43
9	0.0968	0.0390	58
10	0.0991	0.0424	57
11	0.0999	0.0840	49
12	0.1393	0.0419	48
13	0.1397	0.0835	49
14	0.1397	0.0413	56
15	0.1398	0.0841	50
16	0.1414	0.0389	43
17	0.1419	0.0415	50

1.4.3 Leaderboard Analysis

Models trained with all features consistently achieve the best performance, with RMSE values around 0.296–0.301, indicating that combining recipe metadata and review-derived tags provides the strongest predictive signal. When using tag-only features, performance declines moderately but remains competitive, suggesting that review tags capture meaningful information about recipe quality. In contrast, metadata-only models perform substantially worse (RMSE 0.373), highlighting the importance of semantic signals derived from reviews. Differences between architectures are relatively small, indicating that feature representation has a larger impact on performance than network design in this task. Loss function optimization appear to be more model specific than generalizable across the full set of models.

1.5 Hyperparameter Grid Search

After identifying the strongest architectural variants, a targeted **hyperparameter grid search** was performed to refine the final model configuration. Rather than repeating the full architecture sweep, this search focused on the best-performing setup from the leaderboard: a **deep architecture trained with all features using Huber loss**.

The grid search evaluated combinations of three key optimization parameters:

- **Learning Rate:** `[1e-3, 5e-4, 1e-4]`
- **Weight Decay:** `[1e-4, 1e-5, 0.0]`
- **Batch Size:** `[128, 256, 512]`

For each parameter combination, the training pipeline was re-executed with updated configuration values and the model was trained using the same train/validation/test split to ensure consistent evaluation.

Each experiment run was automatically saved with a unique identifier containing the hyperparameter values, enabling results to be easily tracked and integrated into the experiment leaderboard for comparison.

[ ]:
```
hp_leaderboard = generate_hp_leaderboard(s.results_dir)
```

```
=== Hyperparameter Grid Search Leaderboard ===
| Run Date      | Architecture   |     LR |   Batch |   Decay | Ablation     |
Loss   |   RMSE |    MSE |    MAE |   Epochs |
|:--------------|:---------------|-------:|--------:|--------:|:-------------|:-
------|-------:|-------:|-------:|---------:|
| Mar 12, 23:13 | deep           | 0.0001 |     256 |  0      | all_features |
huber  | 0.2967 | 0.088  | 0.2208 |      105 |
| Mar 13, 00:45 | deep           | 0.0005 |     512 |  0      | all_features |
huber  | 0.2969 | 0.0881 | 0.2195 |       77 |
| Mar 13, 00:13 | deep           | 0.0005 |     512 |  0.0001 | all_features |
huber  | 0.297  | 0.0882 | 0.2192 |       66 |
| Mar 12, 20:10 | deep           | 0.0001 |     128 |  1e-05  | all_features |
huber  | 0.2973 | 0.0884 | 0.2202 |       80 |
| Mar 13, 00:01 | deep           | 0.001  |     512 |  0      | all_features |
huber  | 0.2979 | 0.0887 | 0.2206 |       52 |
| Mar 12, 23:40 | deep           | 0.001  |     512 |  0.0001 | all_features |
huber  | 0.2982 | 0.0889 | 0.2182 |       49 |
| Mar 12, 21:58 | deep           | 0.0005 |     256 |  1e-05  | all_features |
huber  | 0.2982 | 0.0889 | 0.22   |       47 |
| Mar 12, 22:12 | deep           | 0.0005 |     256 |  0      | all_features |
huber  | 0.2983 | 0.089  | 0.2179 |       46 |
| Mar 12, 19:33 | deep           | 0.0001 |     128 |  0.0001 | all_features |
huber  | 0.2986 | 0.0892 | 0.2221 |       97 |
| Mar 12, 21:44 | deep           | 0.0005 |     256 |  0.0001 | all_features |
huber  | 0.2993 | 0.0896 | 0.2209 |       46 |
| Mar 12, 22:45 | deep           | 0.0001 |     256 |  1e-05  | all_features |
huber  | 0.2998 | 0.0899 | 0.222  |      101 |
| Mar 12, 20:42 | deep           | 0.0001 |     128 |  0      | all_features |
huber  | 0.3002 | 0.0901 | 0.2233 |       86 |
| Mar 12, 19:03 | deep           | 0.0005 |     128 |  1e-05  | all_features |
huber  | 0.3005 | 0.0903 | 0.22   |       39 |
| Mar 12, 21:33 | deep           | 0.001  |     256 |  0      | all_features |
huber  | 0.3006 | 0.0904 | 0.2192 |       40 |
| Mar 12, 21:13 | deep           | 0.001  |     256 |  0.0001 | all_features |
huber  | 0.3007 | 0.0904 | 0.2185 |       37 |
| Mar 12, 23:51 | deep           | 0.001  |     512 |  1e-05  | all_features |
huber  | 0.3009 | 0.0905 | 0.2216 |       47 |
| Mar 12, 21:23 | deep           | 0.001  |     256 |  1e-05  | all_features |
huber  | 0.301  | 0.0906 | 0.2214 |       39 |
| Mar 12, 19:18 | deep           | 0.0005 |     128 |  0      | all_features |
huber  | 0.3012 | 0.0907 | 0.2205 |       38 |
```

```
| Mar 12, 18:48 | deep           | 0.0005 |     128 |  0.0001 | all_features |
huber  | 0.3013 | 0.0908 | 0.2205 |       39 |
| Mar 12, 18:05 | deep           | 0.001  |     128 |  0.0001 | all_features |
huber  | 0.3019 | 0.0911 | 0.2206 |       35 |
| Mar 12, 18:34 | deep           | 0.001  |     128 |  0      | all_features |
huber  | 0.302  | 0.0912 | 0.2206 |       37 |
| Mar 13, 01:01 | deep           | 0.0001 |     512 |  0.0001 | all_features |
huber  | 0.3021 | 0.0913 | 0.2206 |      113 |
| Mar 12, 18:20 | deep           | 0.001  |     128 |  1e-05  | all_features |
huber  | 0.3038 | 0.0923 | 0.2199 |       35 |
| Mar 13, 01:51 | deep           | 0.0001 |     512 |  0      | all_features |
huber  | 0.3047 | 0.0929 | 0.2247 |       81 |
| Mar 13, 01:27 | deep           | 0.0001 |     512 |  1e-05  | all_features |
huber  | 0.3072 | 0.0944 | 0.2233 |       99 |
| Mar 12, 22:26 | deep           | 0.0001 |     256 |  0.0001 | all_features |
huber  | 0.3288 | 0.1081 | 0.2217 |       69 |
| Mar 13, 00:27 | deep           | 0.0005 |     512 |  1e-05  | all_features |
huber  | 0.3969 | 0.1575 | 0.2222 |       72 |
```

1.5.1 Hyperparameter Grid Search Analysis

The hyperparameter grid search refined the best-performing architecture by evaluating combinations of **learning rate, batch size, and weight decay** for the deep model trained on the full feature set with Huber loss.

The best configuration achieved an **RMSE of 0.2967** using:

- **Learning Rate:** 1e-4
- **Batch Size:** 256
- **Weight Decay:** 0.0

Overall performance across the grid was relatively stable, with most configurations falling within **0.296–0.301 RMSE**, indicating that the model is fairly robust to moderate changes in optimization parameters.

Several patterns emerge from the results. Lower learning rates generally produced more stable outcomes, while very large batch sizes (512) occasionally introduced instability, including a few clear outliers with significantly worse performance. Weight decay had minimal impact on accuracy in this task, suggesting that the model does not suffer heavily from overfitting under the current architecture and dataset size.

Based on these results, the configuration **LR = 1e-4, batch size = 256, weight decay = 0** was selected as the final training setup for the optimized model variants.

1.5.2 Embedding Manifold Comparison Across Architectures

While predictive performance across the full-feature models was very similar (all within ~0.003 RMSE), the learned **latent embedding spaces** differ substantially across architectures. Because the downstream system uses these embeddings for **semantic reranking and similarity search**,

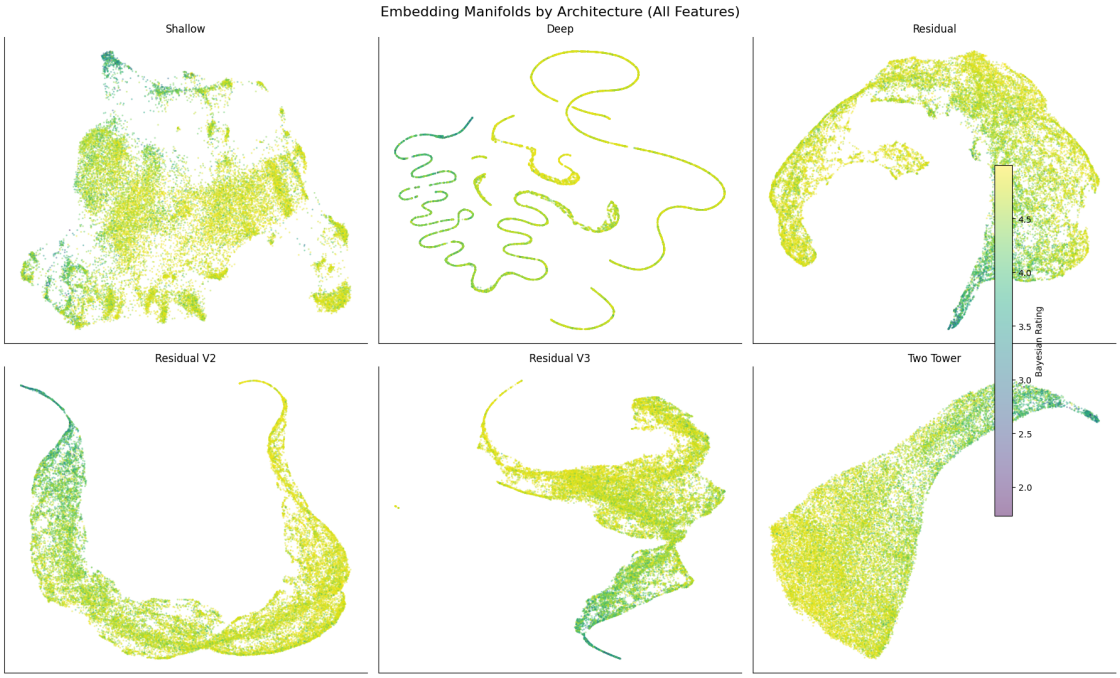
the geometric structure of the embedding space is an important model selection criterion alongside predictive accuracy.

To evaluate the quality of the learned representations, the final embedding layer from each top model variant was extracted and projected into two dimensions using **UMAP**. Each point in the figure represents a recipe from the held-out test set, colored by its Bayesian-smoothed rating. Models that produce smoother manifolds, clearer neighborhood structure, and consistent rating gradients are more suitable for semantic retrieval tasks.

The grid below compares the embedding manifolds produced by six architectures: **Shallow, Deep, Residual, Residual V2, Residual V3, and Two-Tower**. This visualization allows us to assess how different model designs influence the organization of recipe representations in latent space, independent of small differences in predictive error.

[2]:
```
plot_umap_grid()
```

```
c:\Users\iauge\AppData\Local\Programs\Python\Python312\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
c:\Users\iauge\AppData\Local\Programs\Python\Python312\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
c:\Users\iauge\AppData\Local\Programs\Python\Python312\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
c:\Users\iauge\AppData\Local\Programs\Python\Python312\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
c:\Users\iauge\AppData\Local\Programs\Python\Python312\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
c:\Users\iauge\AppData\Local\Programs\Python\Python312\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
C:\Users\iauge\Documents\Drexel MSDS\CS 615\Project\src\visualizations.py:921:
UserWarning: This figure includes Axes that are not compatible with
tight_layout, so results might be incorrect.
  plt.tight_layout()
```



1.5.3 Manifold Analysis

Although predictive performance across architectures differed only marginally, the resulting embedding geometries varied substantially. The Deep architecture collapses the representation into narrow trajectories, suggesting the model primarily learned a one-dimensional rating axis. In contrast, Residual and Two-Tower architectures produce smoother, more continuous manifolds that better preserve local neighborhood structure. Because the embeddings are intended for semantic retrieval and reranking, architectures producing richer manifold geometry are preferred over those optimized solely for predictive accuracy. For this reason I chose to only take the `residual_v2` and `two_tower` models forward into the full inference stage for final comparrison.

1.6 Inference Evaluation

The manifold analysis in the previous section narrowed the candidate architectures to **Residual V2** and **Two-Tower**, both of which produced structured embedding spaces suitable for downstream retrieval tasks. The final step is therefore to compare these two models directly on the inference datasets and determine which architecture should serve as the final scoring model.

The model predicts a **Bayesian-smoothed recipe rating**, bounded approximately between **1 and 5 stars**. Because this score will ultimately be used as a **semantic re-ranking signal within the information retrieval pipeline**, it is important to verify that predictions remain stable, unbiased, and well-calibrated across the rating spectrum.

To make the final decision we examine several complementary diagnostics:

Predicted vs. Actual Ratings This plot compares predicted scores with the true ratings. A well-calibrated model should place points close to the **45° diagonal**, indicating agreement between

predicted and observed values.

Residual Distribution Inspecting the distribution of residuals helps determine whether prediction errors are **centered around zero** and whether large deviations occur systematically or only in isolated cases.

Residuals vs. Actual Ratings Plotting residuals against the true rating allows us to detect **systematic error patterns** across the rating range. Ideally, residuals should be randomly distributed around zero with no clear trend.

Largest Error Inspection Inspect recipes with the largest absolute prediction errors. This provides a qualitative check for **outliers or unusual cases** and helps ensure that large deviations reflect genuinely difficult predictions rather than structural issues in the model.

Spearman Correlation Calculate the monotonic relationship between the predicted and actual values in order to ensure a meaningful relationship between the values, and use the score as a determinent of model quality.

Together, these diagnostics provide a detailed view of model behavior and support the final architecture selection.

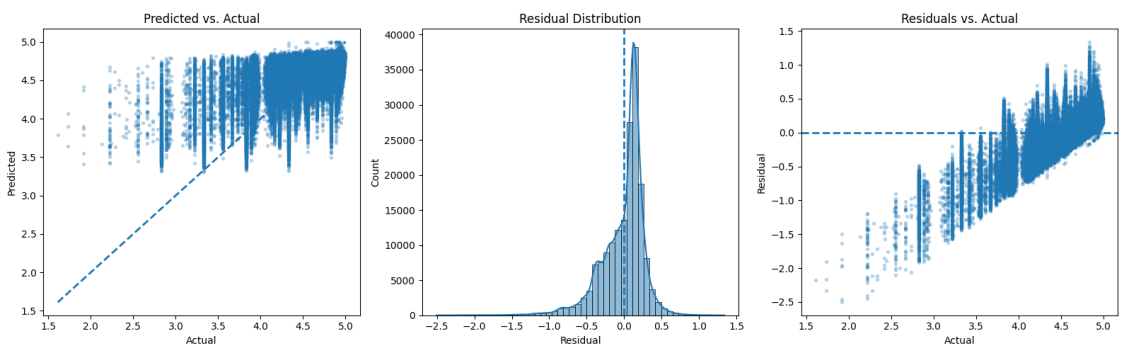
[ ]:
```
# Inference Analysis for Two Tower
two_tower_bundle_path = str(Path(s.best_model_dir) /␣
 ↪"final_two_tower_embeddings.pt")
residual_v2_bundle_path = str(Path(s.best_model_dir) /␣
 ↪"final_residual_v2_embeddings.pt")


print("== Residual Error Summary ==")
print("Residual V2:")
summarize_inference_metrics(residual_v2_bundle_path)
print("\nTwo Tower:")
summarize_inference_metrics(two_tower_bundle_path)
print("\n== Regression Diagnostics ==")
print("Residual V2:")
plot_regression_diagnostics(residual_v2_bundle_path)
print("Two Tower:")
plot_regression_diagnostics(two_tower_bundle_path)
print("\n== Largest Prediction Errors ==")
print("Residual V2:")
res_recipes = show_largest_prediction_errors(residual_v2_bundle_path, top_n=10)
display(res_recipes)
print("Two Tower:")
two_tower_recipes = show_largest_prediction_errors(two_tower_bundle_path,␣
 ↪top_n=10)
display(two_tower_recipes)
print("\n== Spearman Correlation ==")
print("Residual V2:")
calculate_spearmanr(residual_v2_bundle_path)
print("Two Tower:")
```

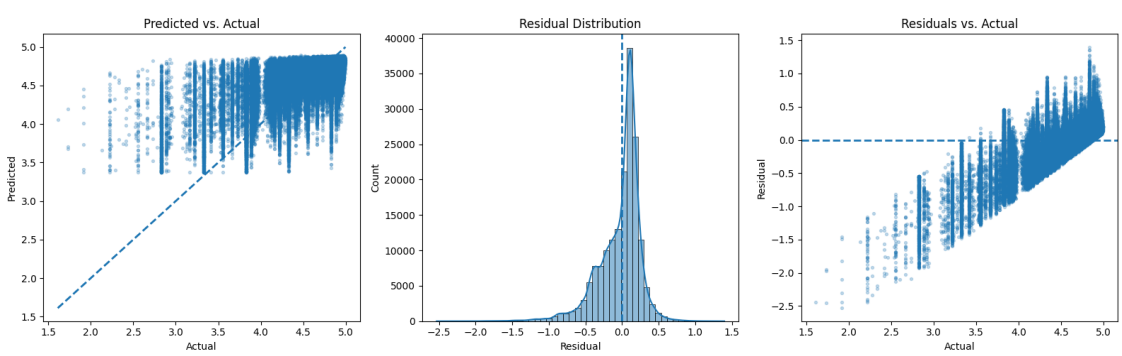
```
calculate_spearmanr(two_tower_bundle_path)
```

```
== Residual Error Summary ==
Residual V2:
|   RMSE |    MAE |    MSE |   Mean Actual |   Mean Predicted |   Mean Residual
|
|-------:|-------:|-------:|--------------:|-----------------:|----------------
:|
|  0.289 | 0.2145 | 0.0835 |         4.616 |           4.6236 |         -0.0076
|
Two Tower:
|   RMSE |    MAE |    MSE |   Mean Actual |   Mean Predicted |   Mean Residual
|
|-------:|-------:|-------:|--------------:|-----------------:|----------------
:|
| 0.2896 | 0.2099 | 0.0838 |         4.616 |           4.6397 |         -0.0236
|

== Regression Diagnostics ==
Residual V2:
```




```
Two Tower:
```



```
== Largest Prediction Errors ==
Residual V2:

                                        recipe_name recipe_id    actual  \
0                                  basic brown rice    407040  1.915375
1                              sushi now sushi rice    261311  1.915375
2                                   filipino lumpia    344469  2.220500
3                your favorite meatloaf with a twist     17571  2.220500
4                              iranian barbari bread    211029  1.732300
5  the worst english muffin pizza    the saddest …    404178  2.276917
6           texas best chili  adopted  not tried yet     45708  2.220500
7             ruby tuesday s buffalo chicken wontons    146404  2.220500
8                                     pecan pie jam     28221  1.610250
9                                 stove top brownies    355387  2.332300

   predicted  residual  abs_error
0   4.419396 -2.504022   2.504022
1   4.372796 -2.457421   2.457421
2   4.672064 -2.451565   2.451565
3   4.624188 -2.403688   2.403688
4   4.066808 -2.334508   2.334508
5   4.605496 -2.328579   2.328579
6   4.435646 -2.215147   2.215147
7   4.399643 -2.179144   2.179144
8   3.786593 -2.176343   2.176343
9   4.502748 -2.170448   2.170448

Two Tower:

                                        recipe_name recipe_id    actual  \
0                                  basic brown rice    407040  1.915375
1                              iranian barbari bread    211029  1.732300
2                              sushi now sushi rice    261311  1.915375
3                                     pecan pie jam     28221  1.610250
4           texas best chili  adopted  not tried yet     45708  2.220500
5                                   filipino lumpia    344469  2.220500
6                your favorite meatloaf with a twist     17571  2.220500
7  the worst english muffin pizza    the saddest …    404178  2.276917
8                 copycat of zaxby s chicken fingers    488933  1.915375
9             ruby tuesday s buffalo chicken wontons    146404  2.220500

   predicted  residual  abs_error
0   4.447463 -2.532088   2.532088
1   4.190826 -2.458527   2.458527
2   4.361280 -2.445906   2.445906
3   4.052187 -2.441937   2.441937
4   4.662365 -2.441865   2.441865
5   4.641028 -2.420529   2.420529
```

```
6   4.625854 -2.405354   2.405354
7   4.610741 -2.333824   2.333824
8   4.216752 -2.301377   2.301377
9   4.517579 -2.297080   2.297080
```

1.6.1 Final Model Selection

Both candidate architectures produce very similar predictive performance.
Spearman correlation on the inference dataset is **0.508 for Residual V2** and **0.505 for the Two-Tower model**, indicating that both models learn a comparable monotonic relationship between predicted and observed ratings. In practical terms, this means either model is capable of producing a usable ranking signal.

However, predictive correlation is not the only objective of this system. The learned embeddings are intended to serve as **semantic representations for retrieval and re-ranking**, making the geometry of the embedding space an important design consideration. The earlier manifold analysis showed that the **Residual V2 architecture produces smoother and more coherent embedding structure**, preserving local neighborhood relationships more effectively than the Two-Tower representation.

Because the predictive metrics are nearly identical, the decision can reasonably be guided by representation quality. The **Residual V2 model provides slightly stronger predictive correlation while also yielding richer embedding geometry**, making it the most suitable architecture for downstream retrieval tasks.

For these reasons, **Residual V2 is selected as the final model** used for full inference and embedding generation.

1.7 Final Embedding Space for Residual_V2

To examine the representation learned by the final model, the penultimate-layer recipe embeddings were projected into two dimensions using **UMAP**. Each point represents a recipe in the full corpus, colored by its Bayesian-smoothed rating.

This projection reveals a large, continuous manifold representing the structure of the recipe corpus in the learned embedding space. High-quality recipes (dark red) concentrate along the outer arc of the manifold, forming dense neighborhoods where many highly rated recipes share similar feature representations. Moving inward and downward along the structure, the color gradient gradually shifts toward lighter tones, indicating recipes with lower Bayesian-smoothed ratings.

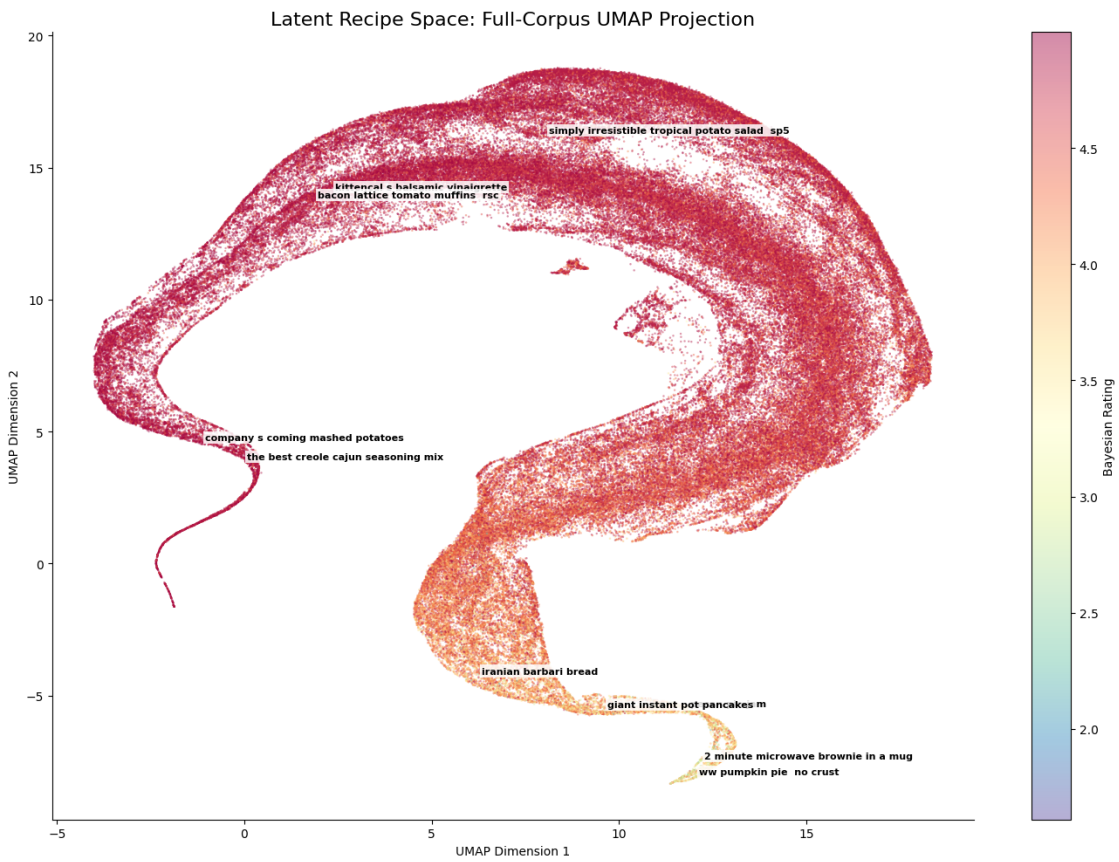
One notable feature is the elongated lower branch of the manifold, where lower-rated recipes appear more frequently. This suggests that the model separates recipes associated with weaker review signals into distinct regions of the representation space. At the same time, the dense clustering along the upper arc indicates that many well-reviewed recipes share similar semantic characteristics.

Overall, the embedding space forms a smooth and coherent structure rather than fragmented clusters, indicating that the residual architecture has learned a continuous representation of recipe quality and similarity. Because these embeddings will later be reused for **semantic re-ranking within the information retrieval system**, this structured geometry provides confidence that nearby recipes in the embedding space correspond to meaningfully related recipes in the corpus.

[20]:
```
projection_path = str(Path(s.best_model_dir) /␣
↪"final_residual_v2_umap_projection.npy")

plot_recipe_manifold(
    bundle_path=residual_v2_bundle_path,
    projection_path=projection_path,
    save_projection=True
)
```

```
c:\Users\iauge\AppData\Local\Programs\Python\Python312\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
```



[20]:
```
array([[14.271424  , 14.931443  ],
       [15.165576  ,  9.121739  ],
       [10.627417  , 12.977495  ],
       …,
       [11.426687  , 13.481635  ],
       [-0.02358997,  2.5139222 ],
```

```
[ 7.1240563 , -5.0676045 ]], dtype=float32)
```

1.8 Conclusion

This project explored how neural network architecture influences both predictive performance and representation quality when modeling recipe ratings from structured metadata and review-derived signals. While shallow, deep, residual, and two-tower architectures produced broadly similar regression accuracy, their learned embedding geometries differed substantially. Residual connections produced smoother and more coherent latent manifolds, preserving local neighborhood structure that is valuable for downstream retrieval tasks.

After comparing candidate architectures using diagnostic metrics and full-corpus inference analysis, the **Residual V2 model** was selected as the final architecture. It achieved slightly stronger monotonic alignment with true ratings while also producing a well-structured embedding space across the full recipe corpus. The resulting embeddings capture meaningful variation in recipe quality and similarity, forming a continuous latent space suitable for semantic operations.

These embeddings can now be reused as a **semantic re-ranking signal within the information retrieval pipeline**, enabling search results to prioritize recipes that are not only textually relevant but also structurally similar to highly rated recipes in the dataset.